Report — Assignment 4: Dynamic Programming and String Algorithms

---

1. Experiment Design Card (Filled BEFORE the first run)

Field	Content
Question	Edit Distance: Does the cost grow as theory suggests: recursion — exponentially, table and two rows — as n·m? What does the two-row version gain/lose? String Search: How much worse is Naive than KMP on the worst-case input, and do they perform similarly on random text?
Sizes (n)	Recursion: n = 4, 6, 8, 10, 12 (lengths of both strings) — beyond this, time grows too fast. Table / two rows: n = 250, 500, 1000, 2000, 4000 (max = 16 × min; 4000×4000 = 16 million cells, takes seconds not hours). Search: text length n = 100,000; 200,000; 400,000; 800,000; 1,600,000, pattern length m = 50 (fixed).
Inputs	Edit Distance: random (alphabet of 4 letters) — average case; similar (b = a with ~10% mutations) — similar words, like typos in addresses; disjoint (a = "aaa…", b = "bbb…") — worst case for recursion (no character matches, maximum branching). Search: random (both text and pattern are random, 4-letter alphabet) — average case; worst (text "AAA…A", pattern "AA…AB") — worst case for Naive: almost the entire pattern is compared at every position.
Metrics	Edit Distance: ops — number of recursive calls (A) or computed cells (B, C), tracked inside the algorithm; mem_cells — number of allocated memory cells (B: (n+1)(m+1), C: 2(m+1)); time_ms — median of 5 runs (3 for recursion). Search: ops — number of character comparisons (in KMP, includes LPS array construction); time_ms. Counters are independent of the laptop's hardware, making them the primary proof.
Prediction	Recursion: the number of calls grows exponentially (roughly ×5–6 for n+1 on disjoint, less on random), n=12 is already noticeably longer than 1 sec. B and C: ops = n·m exactly, doubling n increases time by ×4; B and C should have similar execution times, memory usage B ~ n·m, C ~ 2m. Input type (random/similar/disjoint) does not affect ops. Search: on random Naive ≈ 1.3·n comparisons, KMP ≈ 1–2·n, i.e., almost identical; on worst Naive ≈ n·m = 50·n, KMP ≤ 2·n (linear growth).

Plots	1) recursive calls vs n, log-axis — is the growth exponential? 2) execution time B and C vs n — is the growth quadratic and is there a difference? 3) memory (cells) B vs C vs n, log-axis — what is the benefit of two rows? 4) comparisons Naive/KMP on random and worst vs n — is it linear or not? 5) ratio of comparisons to n — how many comparisons per text character.

Card changes after initial tests: added one size n = 14 for recursion, solely on the disjoint input (to actually measure the recursion limit rather than extrapolating). Everything else remains as in the card. All data: results/results.csv (1 row = 1 run, seed = 42 + n, counters matched perfectly upon rerun).

2. Plots and Results

Plot 1 (1_recursive_calls.png). On the log-axis, the lines are straight — meaning the growth of calls is exponential; disjoint at n=12 yields $3.8 \cdot 10^8$ calls, random yields $8.7 \cdot 10^6$.

Plot 2 (2_dp_time.png). The execution time of B and C grows approximately quadratically, and the input type has almost no effect; at n=4000, the two-row version is noticeably faster than the table (≈74 ms vs ≈127 ms on random).

Plot 3 (3_dp_memory.png). The table occupies ~n^2 cells (16 million at n=4000), while the two rows occupy ~2n (8,002); the gap increases with n.

Plot 4 (4_search_comparisons.png). Naive on the worst input lies about 25–50 times higher than the other lines; the other three are close to n.

Plot 5 (5_search_ratio.png / ratio). KMP performs ≤ 2 comparisons per text character (1.25 on random, 2.0 on worst). Naive on worst is ≈ 50 (= m), and on random it is 1.33.

Plot 6 (6_search_time.png). Execution time follows the operation counters and is linear with respect to n; on worst, Naive takes ≈ 136 ms compared to ≈ 5.1 ms for KMP at n = 1.6 million.

Key Metrics (Medians)

Algorithm	Input	n	ops	time_ms
recursive	disjoint	12 / 14	$3.77 \cdot 10^8$ / $1.19 \cdot 10^{10}$	866 / 25,633
recursive	random	12	$8.7 \cdot 10^6$	23
table	random	4000	$1.6 \cdot 10^7$	127
tworows	random	4000	$1.6 \cdot 10^7$	74
naive	worst	$1.6 \cdot 10^6$	$8.0 \cdot 10^7$	136
kmp	worst	$1.6 \cdot 10^6$	$3.2 \cdot 10^6$	5.1

3. Prediction vs Result

Edit Distance

• **Recursion:** Expected exponential growth (×5–6 for n+1 on disjoint) — observed ×5.3–5.6 per step (12→14: ×31.5 over two steps), which matches the prediction. Expected n=12 to be «noticeably longer than 1 s» — on disjoint it took 0.87 s (slightly less), and on random just 23 ms, meaning I overestimated the random case. Surprise: on random, the number of calls is not monotonically increasing with n (n=8: 54,518, n=10: 42,684). Reason: for each n, a unique pair of strings is generated, and the number of calls heavily depends on how many of the ending characters happen to match (a match directs the recursion down a single branch). On similar, there are almost no recursive branching calls (5–657) for the same reason.

• **Table and Two Rows:** ops = n·m exactly (62,500 → 16,000,000) and is independent of the input type — this perfectly matched. Execution time roughly increases by ×4 when n doubles (noisy: 3.4 and 4.8 for the table on random). I expected B and C to have similar times, but for n ≥ 2000 the two-row version is faster (≈1.7× faster at n=4000). Probable explanation (not tested independently): a full table at n=4000 occupies ~64 MB and does not fit in the CPU cache, whereas two rows occupy around 32 KB.

String Search

Everything matched the prediction: Naive/random 1.33·n, KMP/random 1.25·n, Naive/worst ≈ 50·n, KMP/worst ≈ 2·n. Surprise: on random, KMP is not faster than Naive (9.7 vs 8.7 ms at n=1.6 million), even though it executes fewer comparisons. This is likely because KMP has higher overhead per step (accessing the LPS array), and the difference in comparisons is only 6%; at such small execution times (1–10 ms), system noise also plays a significant role.

4. Short Answers

Which input made the search the slowest and why?

worst: text consisting entirely of «A»s, and a pattern of 49 «A»s followed by a «B». Naive compares 49 characters at every one of the ~n positions and fails on the last one, resulting in ≈ 50·n comparisons ($8 \cdot 10^7$ at n=1.6 million, taking 136 ms compared to 8.7 ms on random). This is also the hardest input for KMP, but KMP is strictly bounded to 2 comparisons per character (5.1 ms), because the text index is never decremented.

What n did recursion handle and what stopped it?

Measured up to n=14 on disjoint: taking 25.6 seconds and $1.2 \cdot 10^{10}$ calls; n=12 took 0.87 seconds. Every +1 to n results in roughly ×5.6 time increase, meaning (by extrapolation, not measured) n=15 would take ≈ 2–3 minutes, n=16 ≈ 15 minutes, which is why I stopped at 14. I was stopped by execution time, not memory: the maximum recursion depth is merely n+m. The limit heavily depends on the input: on random, n=12 only took 23 ms.

What does the two-row method gain and lose compared to the full table?

Gain — memory: at n=4000 it allocates 8,002 cells instead of 16,008,001, and in our benchmark, it also ran faster (≈74 ms vs ≈127 ms). Loss: previous rows are overwritten, so you cannot reconstruct the actual edit operations (which characters to insert/replace) from this solution, only the final distance value; full edit path reconstruction requires the entire table (or a more complex divide-and-conquer scheme).

Which search algorithm should be chosen for a city platform?

KMP. On random text, it performs no worse than Naive (the difference is within the noise margin), but on repetitive data (logs often contain long identical sequences) it guarantees ≤ 2·n comparisons, whereas Naive can be 25 times slower on such inputs. The cost is slightly more code (the LPS array construction). For short street names, the difference is negligible, but the worst-case performance guarantee is far more important than simplicity.

5. Note on Time Measurements

The measured execution time is the median of 5 runs (3 for recursion) following a warm-up phase, ranging from microseconds up to 25 seconds. At values between 1–10 ms, noise is visible (for example, KMP/random on n=100,000 takes longer than on n=200,000: 1.8 vs 1.2 ms), which is influenced by the JIT compiler, garbage collector, and OS background processes. Therefore, time is not the sole proof: the internal counters (ops, mem_cells) are tracked directly inside the algorithms, are identical on every deterministic run, and act as the primary evidence confirming conclusions about growth rates and worst-case scenarios; time simply validates that these counters reflect actual real-world speed.